

Autonomous Driving Vehicle

Contents

1	Hardware	2
2	Software	6
2.1	Architecture	6
2.1.1	Embedded Platform	6
2.1.2	Brain	6
2.2	Embedded Platform	6
2.2.1	Peripheral	6
2.2.2	Sensor	8
2.2.3	Actuator	10
2.3	Brain	11
2.3.1	Module Communication	11
2.4	IO	11
2.4.1	Perception	11
2.4.2	Localization	12
2.4.3	Planning	12
2.4.4	Control	12
2.5	Monitor	12
2.6	Simulation	12
3	Source & Description	13

1 Hardware

Vehicle Platform

STM32F401 NUCLEO

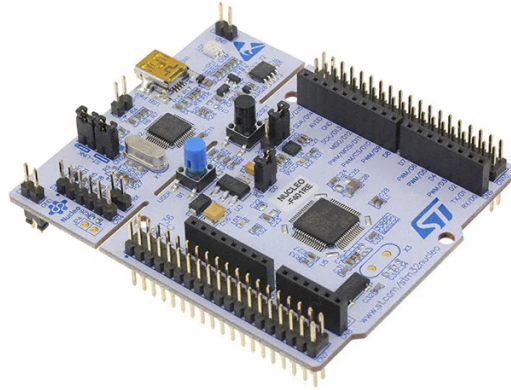


Figure 1: STM32F401 NUCLEO board

Property	Value
MCU	STM32F401RE (ARM Cortex-M4)
Clock	Up to 84 MHz
Flash	512 KB
RAM	96 KB
Interface	UART, I2C, SPI, GPIO, ADC, PWM
Role	Low-level control: motor, servo, sensor reading

Table 1: STM32F401 NUCLEO specifications

Raspberry Pi 5



Figure 2: Raspberry Pi 5

Property	Value
CPU	Broadcom BCM2712, quad-core Cortex-A76 @ 2.4 GHz
RAM	4 GB / 8 GB LPDDR4X
OS	Ubuntu 24.04 (64-bit)
Interface	UART, USB, CSI camera, Ethernet, Wi-Fi
Role	High-level brain: perception, planning, control

Table 2: Raspberry Pi 5 specifications

Encoder: AS5600

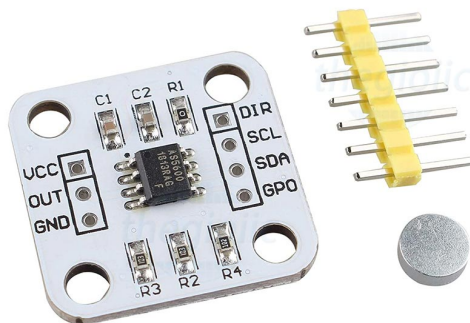


Figure 3: AS5600 magnetic rotary encoder module

Property	Value
Type	Contactless magnetic rotary encoder
Resolution	12-bit (4096 positions/revolution)
Interface	I2C (address 0x36)
Supply voltage	3.3 V – 5 V
Output	Absolute angle 0°–360°
Role	Measures wheel rotation for speed estimation

Table 3: AS5600 encoder specifications

IMU: BNO055

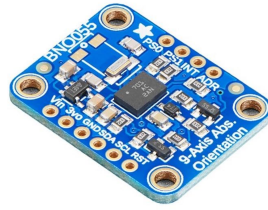


Figure 4: BNO055 9-axis IMU module

Property	Value
Axes	9-axis (accelerometer + gyroscope + magnetometer)
Interface	I2C (address 0x28 / 0x29)
Output	Euler angles: yaw, pitch, roll (degrees)
Supply voltage	3.3 V
Fusion mode	NDOF (on-chip sensor fusion)
Role	Provides heading angle (yaw) for state estimation

Table 4: BNO055 IMU specifications

Actuator: Steering Servo



Figure 5: 180° steering servo

Property	Value
Type	PWM position servo
Range	0°–180°
Control	PWM 50 Hz, pulse width 1–2 ms
Supply voltage	5 V
Role	Controls front wheel steering angle

Table 5: Steering servo specifications

Simulator

- Encoder: AS5600
- Motor
- 3D Model design

2 Software

2.1 Architecture

2.1.1 Embedded Platform

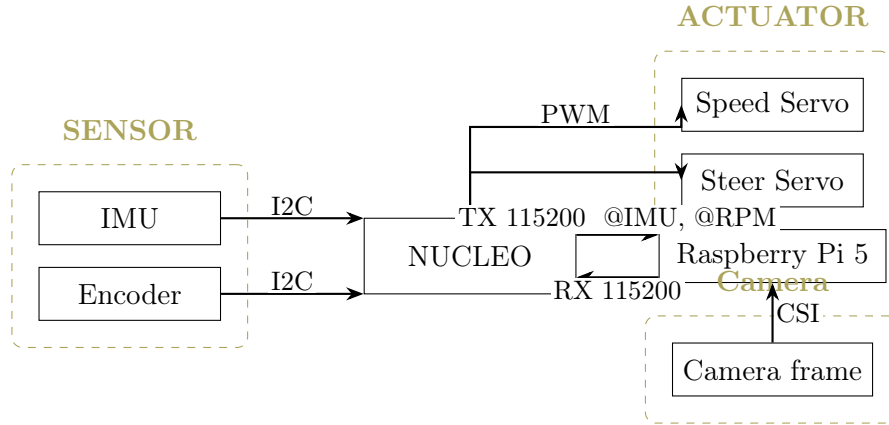


Figure 6: Embedded Platform: NUCLEO reads sensors (I2C), drives actuators (PWM), communicates with Raspberry Pi 5 via UART

2.1.2 Brain

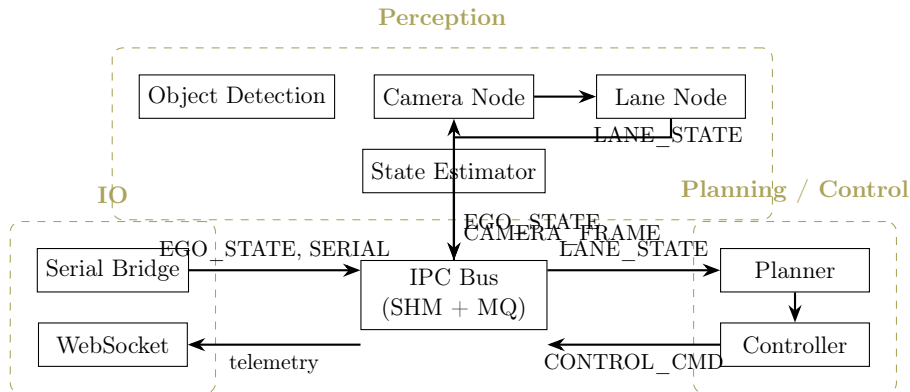


Figure 7: Brain architecture: nodes communicate via IPC bus (SHM for latest-wins, MQ for queued messages)

2.2 Embedded Platform

2.2.1 Peripheral

I2C (Inter-Integrated Circuit)

I2C is a two-wire synchronous serial protocol using a shared clock line (SCL) and a bidirectional data line (SDA). Communication is initiated by a START condition (SDA falls while SCL is high), followed by a 7-bit device address, a read/write bit, an ACK from the slave, data bytes, and a STOP condition (SDA rises while SCL is high). In this project, I2C is used to communicate with the BNO055 IMU at address 0x28.

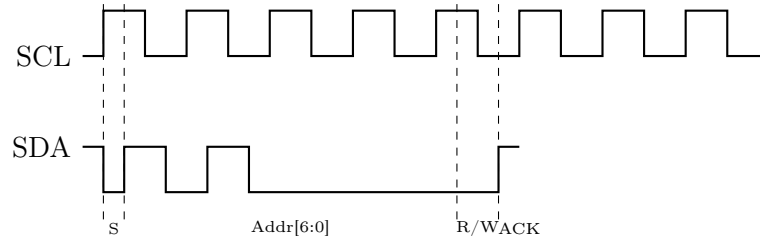


Figure 8: I2C transaction: START, 7-bit address (0x28), R/W bit, ACK

PWM (Pulse Width Modulation)

PWM generates a periodic digital signal with fixed frequency and variable duty cycle. The STM32 timer compares the counter value against a capture/compare register (CCR); output is high when $CNT < CCR$ and low otherwise. PWM is used to control the servo steering angle and to send throttle commands to the ESC. Typical servo period is 20 ms (50 Hz); pulse width 1–2 ms maps to 0° – 180° .

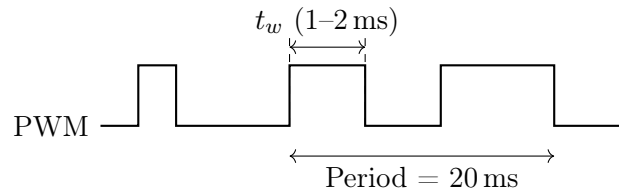


Figure 9: PWM signal: varying pulse width, fixed 50 Hz period

UART (Universal Asynchronous Receiver/Transmitter)

UART is an asynchronous serial protocol with no shared clock. A frame consists of: 1 start bit (low), 8 data bits (LSB first), optional parity, and 1–2 stop bits (high). Both ends must agree on the same baud rate. This project uses 115200 baud, 8N1 (no parity, 1 stop bit). The STM32 transmits sensor data to the Raspberry Pi 5 over UART in ASCII frames: `@IMU:yaw,pitch,roll;;\r\n` and `@RPM:rpm;;\r\n`.

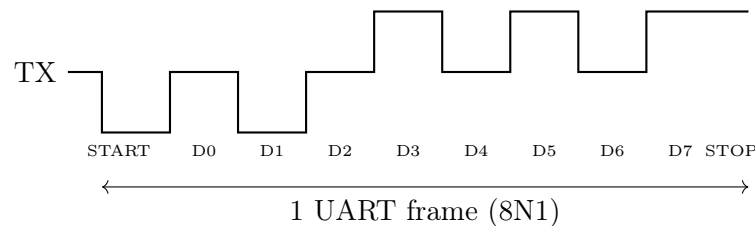


Figure 10: UART frame: start bit, 8 data bits (LSB first), stop bit

RCC (Reset and Clock Control)

The RCC peripheral on the STM32F401 manages system clock configuration and peripheral clock gating. The internal HSI oscillator (16 MHz) or external HSE crystal can be fed into the PLL to generate the system clock (SYSCLK). On the STM32F401 NUCLEO, SYSCLK is configured to 84 MHz. Peripheral buses APB1 (max 42 MHz) and APB2 (max 84 MHz) are derived via prescalers. Each peripheral must have its bus clock explicitly enabled via `RCC_APBxENR` before use.

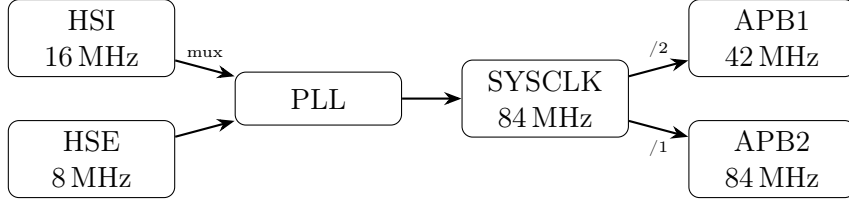


Figure 11: STM32F401 clock tree (simplified): HSI/HSE $\rightarrow$ PLL $\rightarrow$ SYSCLK $\rightarrow$ APB buses

Timer

STM32 general-purpose timers (TIM2–TIM5) use an up-counter that increments each tick. The tick frequency is $\frac{f_{APB} \times 2}{PSC + 1}$; the counter resets at **ARR** (Auto-Reload Register), producing an overflow period of $\frac{ARR + 1}{f_{tick}}$. In this project:

- **TIM1/TIM4** — PWM output for servo and ESC ($f = 50$ Hz, $ARR = 19999$, $PSC = 83$)
- **TIM2** — encoder interface mode, counts quadrature edges from AS5600 (via GPIO simulation)
- **TIM3** — periodic interrupt for UART transmit scheduling (1 kHz)

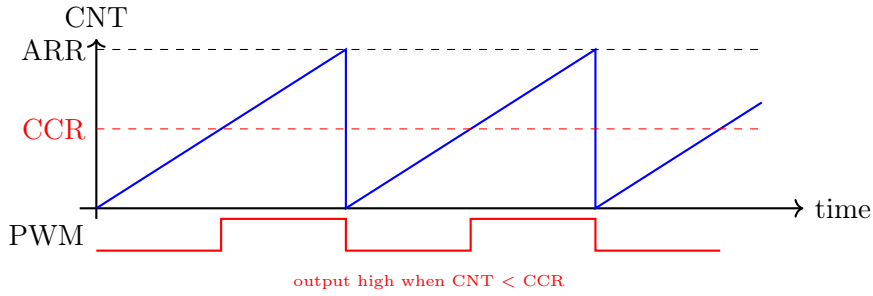


Figure 12: Timer up-count mode: counter ramps to **ARR**, resets; PWM output high while $CNT < CCR$

2.2.2 Sensor

IMU: BNO055

*Communication protocol: I2C, address **0x28** (ADDR pin low) or **0x29** (ADDR pin high). Bus speed up to 400 kHz (Fast Mode). The MCU acts as master; BNO055 is slave.*

Key registers:

Register	Address	Description
CHIP_ID	0x00	Should read 0xA0 — used to verify communication
OPR_MODE	0x3D	Operation mode: 0x0C = NDOF (full fusion), 0x00 = CONFIG
PWR_MODE	0x3E	Power mode: 0x00 = Normal
SYS_TRIGGER	0x3F	System trigger: 0x20 = reset
UNIT_SEL	0x3B	Unit selection: bit0=0 → degrees, bit0=1 → radians
EUL_H_LSB	0x1A	Euler heading (yaw) LSB — 1 LSB = 1/16°
EUL_H_MSB	0x1B	Euler heading (yaw) MSB
EUL_P_LSB	0x1C	Euler pitch LSB
EUL_P_MSB	0x1D	Euler pitch MSB
EUL_R_LSB	0x1E	Euler roll LSB
EUL_R_MSB	0x1F	Euler roll MSB
CALIB_STAT	0x35	Calibration status: bits [7:6]=sys, [5:4]=gyro, [3:2]=accel, [1:0]=mag (3=fully calibrated)

Table 6: BNO055 key registers

Initialization & data acquisition sequence:

1. **Power-on wait:** delay ≥ 650 ms after power-up for internal oscillator to stabilize.
2. **Verify chip:** read CHIP_ID (0x00) → expect 0xA0; retry up to 5 times if NACK.
3. **Enter CONFIG mode:** write 0x00 to OPR_MODE (0x3D); wait 19 ms.
4. **Set units:** write UNIT_SEL (0x3B) → 0x00 (degrees, m/s², dps).
5. **Set power mode:** write 0x00 (Normal) to PWR_MODE (0x3E).
6. **Set NDOF fusion mode:** write 0x0C to OPR_MODE; wait 7 ms.
7. **Read Euler angles** (burst read 6 bytes from 0x1A):
 - Combine LSB+MSB: `int16_t raw = (MSB << 8) | LSB`
 - Convert: `float degrees = raw / 16.0f`
 - Order: [EUL_H, EUL_P, EUL_R] → yaw, pitch, roll
8. **Repeat step 7** at desired rate (project uses 100 Hz polling via TIM3 interrupt).

Encoder: AS5600

Communication protocol: I2C, fixed address 0x36 (not configurable). Bus speed up to 1 MHz (Fast Mode Plus). Provides 12-bit absolute angle output.

Key registers:

Register	Address	Description
ZMCO	0x00	Number of times ZPOS/MPOS have been permanently written
ZPOS_H	0x01	Zero position (start angle) high byte [11:8]
ZPOS_L	0x02	Zero position (start angle) low byte [7:0]
CONF_H	0x07	Configuration high byte (slow filter, fast filter, watchdog)
CONF_L	0x08	Configuration low byte (power mode, hysteresis, output stage, PWM freq)
RAW_ANGLE_H	0x0C	Raw angle high byte [11:8] (before zero-position offset)
RAW_ANGLE_L	0x0D	Raw angle low byte [7:0]
ANGLE_H	0x0E	Processed angle high byte [11:8] (after zero offset, scaled to MANG)
ANGLE_L	0x0F	Processed angle low byte [7:0]
STATUS	0x0B	bits [5:3]: MH=magnet too strong, ML=magnet too weak, MD=magnet detected
AGC	0x1A	Automatic gain control value (0–255); nominal 128
MAGNITUDE_H	0x1B	CORDIC magnitude high byte
MAGNITUDE_L	0x1C	CORDIC magnitude low byte

Table 7: AS5600 key registers

Initialization & data acquisition sequence:

1. **Verify magnet:** read STATUS (0x0B), check bit MD (bit 5) = 1 → magnet detected. If ML or MH set, reposition magnet until MD only.
2. **Check AGC:** read AGC (0x1A); value should be 50–200 for reliable operation.
3. **(Optional) Set zero position:** write current angle to ZPOS_H/L to define the zero reference.
4. **Read angle** (burst read 2 bytes from 0x0E):
 - `uint16_t raw = ((ANGLE_H & 0x0F) << 8) | ANGLE_L`
 - Range: 0–4095 → 0°–360°
 - Convert: `float deg = raw * 360.0f / 4096.0f`
5. **Compute RPM:** track angle over time, handle 0°/360° wrap-around, compute $\Delta\theta/\Delta t$, convert to RPM.
6. **Repeat step 4–5** at desired rate (project uses 1 kHz via TIM3 interrupt).

2.2.3 Actuator

Steering: Angle 180 Servo

- ...

Motivation: Brushless Motor & ESC

- ...

2.3 Brain

2.3.1 Module Communication

Kernel

...

Inter Process Communication

- Message Queue — ...
- Unix Domain Socket — ...
- TCP Socket — ...
- Shared Memory — ...

Pub/Sub Model

- Node — ...
- Topic — ...
- Publish — ...
- Subscribe — ...

Drop Policy

- Topic
- Message
- Payload

C API / C++ Binding

Python Binding

2.4 IO

- Serial Bridge
- Web Server / Web Socket

2.4.1 Perception

Camera

- Input
- Output
- Algorithm

Lane Detection

- Input
- Output
- Algorithm

Object Detection

- Input
- Output
- Algorithm

State

- Input
- Output
- Algorithm

2.4.2 Localization

- Input
- Output
- Algorithm

2.4.3 Planning

- Input
- Output
- Algorithm

2.4.4 Control

- Input
- Output
- Algorithm

2.5 Monitor

- Web Server
- Dashboard

2.6 Simulation

- Visualization
- Direction Simulation

3 Source & Description